Service boundary and Responsibility analysis

Service Map

Module	Purpose	Inputs	Outputs	Dependencies	Boundary	Recommendation
`frontend`	Main product UI for chat, RAG, workflows, notebooks, settings	Browser events, auth token, backend APIs	UI state, API calls, persisted client state	React, Zustand, React Query, backend API	`Blurred` because it mixes page features, local stores, and direct API usage	Keep independent, but split into feature packages and remove duplicate logic from `landing-page`
`landing-page`	Marketing site	Public browser traffic, survey submits	Marketing pages, CTA flows, survey POSTs	React/Vite, some backend APIs	`Blurred` because it contains near-copy product modules not used by the actual app shell	Keep independent only as a slim marketing app; merge/shared-package the duplicated product code
`backend` composition root	Boots Fastify, DB, Redis, queues, domains	Env/config, infra connections	Running API server	Fastify, Postgres, Redis, workers	`Mostly clean` at top level	Keep independent
Auth & identity	Register/login/refresh/profile/password/OAuth/2FA bootstrap	Auth HTTP requests, user creds, JWTs	Tokens, user profiles, auth state changes	DB, bcrypt, JWT, Google auth, Redis blacklist	`Blurred` because one route owns too much identity/security logic	Keep independent, but split OAuth, credential auth, session/token, and 2FA
Access & entitl	Role/tier/workspace permissions and limits	User role, subscription tier,	Effective permissions/ limits	Access-control service,	`Moderately clean` conceptually, but	Keep independent, refactor into a real

ements		workspace mode		subscription middleware, preferences	spread across middleware/routes/services	entitlement service
Chat orchestration	Session lifecycle, message persistence, streaming completions, cost tracking, training signals	Chat requests, model params, session IDs, user API keys	Assistant messages, SSE stream, session/message records, usage data	LLM providers, DB, RAG, cost calc, subscription checks	`Blurred` and overloaded	Keep independent, split session management, inference orchestration, and training telemetry
Knowledge/file ingestion	Upload, validate, store, process, reprocess, delete documents	Multipart files, metadata, user context	File records, stored files, queue jobs, processed chunks	Storage provider, queue, document processor, embeddings, DB	`Blurred` and overloaded	Keep independent, split storage API from indexing pipeline
Retrieval/RAG	Search, hybrid retrieval, query enhancement, analytics, grounded chat, RAG sessions	Queries, file IDs, session IDs, chat prompts	Ranked chunks, RAG chat responses, analytics, search history	RAG services, cache, embeddings, DB, LLMs	`Very blurred` and heavily overloaded	Keep independent, but split retrieval, RAG chat, analytics, and session/history
Model catalog	List/filter available models and defaults	Filter params	Model metadata and defaults	Models service, cache, DB	`Clean`	Keep independent
LLM provider gateway	Call OpenAI/Anthropic/Groq/etc. and stream responses	Messages, chosen model, params, user/platform keys	Normalized model responses/streams	External LLM SDKs, config	`Moderately clean` but mixed with provider specifics	Keep independent; make it the single inference gateway

Workflow engine	Persist and execute graph-based workflows	Workflow graphs, execution input, user keys	Execution logs, outputs, workflow records	DB, workflow executor, LLM/RAG services	`Moderately clean` but tightly coupled to chat/RAG internals	Keep independent
Notebooks	Notebook CRUD and cell execution with AI/RAG	Notebook/cell requests, attached docs	Notebook/cell state, execution outputs	DB, LLM providers, RAG	`Blurred` because execution is embedded in route logic and reuses chat/RAG ad hoc	Keep independent, but factor out a notebook runtime
Preferences / user settings / security	User prefs, API keys, security settings, 2FA sessions	Settings/security HTTP requests	Updated prefs, masked API keys, security state	DB, encryption, authenticator/QR code	`Somewhat blurred` because account management is split awkwardly across routes	Merge into one account-management bounded context with submodules
Feedback & training signals	Message feedback and training consent	Ratings, consent toggles	Feedback records, consent state	DB, auth	`Clean`	Keep independent
Bookmarks	Save/retrieve bookmarked entities	Bookmark CRUD requests	Bookmark records/counts	DB, auth	`Clean`	Keep independent but low priority
Survey / product discovery	Collect market feedback from landing/demo flows	Survey form payloads	Survey records/analytics	DB	`Clean`, but not core-platform	Keep separate from core AI platform; treat as growth/CRM module

Policy	Evaluate policy decisions and emit policy events	Policy evaluation requests	Allow/deny decision + event	Policy service, domain events	`Clean boundary, weak implementation`	Keep independent, but replace stubbed default-allow logic
Ops / telemetry / admin	Health, request logging, admin DB fixes, metrics	Infra checks, admin actions, emitted events	Health responses, logs, maintenance actions	DB, Redis, telemetry hooks	`Blurred/incomplete`	Split health/metrics/admin; register metrics properly
Shared contracts / events / recovery	Shared types, SDK, event contracts, backup/recovery/data-consistency scaffolding	Cross-service imports, event definitions	Shared types/events/helpers	TS packages, Redis/pg libs	`Blurred` because runtime code and aspirational ops code live together; contracts are duplicated	Keep shared contracts; move backup/recovery to ops and dedupe domain events

Evidence That Drives This
The backend “domains” are mostly thin wrappers over very large route files, so the real business boundaries live in routes, not domain modules: [index.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/index.ts#L1), [domains/index.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/domains/index.ts#L1), [chat.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L1), [files.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L1), [rag.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L1). The landing app’s shell is just marketing, but the workspace still carries almost the full product surface: [landing App](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/landing-page/src/App.tsx#L1), [landing auth context](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/landing-page/src/contexts/AuthContext.tsx#L1), [frontend auth context](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx#L1). Policy is stubbed to default allow: [policy-service.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/policy-service.ts#L1). Metrics exists but is not wired into domain registration: [metrics.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/

metrics.ts#L1),
[domains/index.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/domains/index.ts#L1). Domain-event contracts are duplicated: [backend domain-events](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/utils/domain-events.ts#L1), [shared sdk domain-events](/Users/aeishwary/JoaLLM-platform/joallm-platform/shared/sdk/src/events/domain-events.ts#L1).

Answers

Current service boundaries are only partially aligned with an AI platform.

At the deployment level, `frontend`, `backend`, and `landing-page` are sensible. Inside the platform, though, the boundaries are not AI-platform-native yet. The code splits by HTTP route more than by durable platform capabilities like inference, retrieval/indexing, evaluation, identity, and workspace management.

Services that should exist but are currently missing:
- A real `Inference Gateway` that owns provider routing, fallback, retries, quotas, and model policy.
- A separate `Indexing Pipeline` service for document extraction/chunking/embedding lifecycle instead of burying it under file routes.
- An `Evaluation/Observability` service for prompt quality, RAG quality, latency/cost analysis, traces, and experiment tracking.
- A `Prompt/Template Registry` for reusable prompts, system instructions, and versioning.
- A `Workspace/Tenant Collaboration` service for teams, shared assets, and org-level governance.
- A real `Billing/Payments` service; subscriptions currently look mostly entitlement-focused rather than payment-integrated.
- A real `Policy Engine`; the current one is effectively a placeholder.
- If agents are strategic, an `Agent Runtime / Task Orchestration` service separate from workflows and notebooks.

Services/modules currently overloaded:
- `RAG` is the most overloaded: search, hybrid ranking, query enhancement, grounded chat, analytics, suggestions, reindexing, and session history all live together in [rag.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L1).
- `Chat` is also overloaded: sessions, messages, streaming inference, title generation, usage/cost tracking, and message signals in [chat.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L1).
- `Files` owns too much: upload validation, storage, processing orchestration, queue fallback, indexing handoff, download, and admin-ish maintenance in [files.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L1).

- `Auth` carries too many responsibilities across identity, sessioning, OAuth, and security in [auth.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L1).
- `landing-page` is overloaded organizationally because it contains a second quasi-product app instead of a focused marketing surface.

Net: the repo has the right major capabilities for an AI platform, but the internal boundaries should be refactored around core platform domains: `identity`, `entitlements`, `inference`, `knowledge ingestion`, `retrieval`, `conversation`, `workflow/agent runtime`, `notebook runtime`, `evaluation/observability`, and `workspace governance`.

Review this codebase for responsibility leakage.

Identify where:

- API layers contain business logic

- Business logic contains infrastructure logic

- Data access is mixed with orchestration

- Frontend is carrying backend assumptions

- AI-specific logic is scattered instead of centralized

For each issue:

- Show the likely design smell

- Explain why it is a problem

- Suggest a better responsibility split

Findings

1. **API routes are acting as application services**

Files like
[chat.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L15),
[rag.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L17),
[files.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L36), and
[auth.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L36) do validation, authorization decisions, DB reads/writes, orchestration, cost tracking, and response shaping in one place.

Likely smell: `Fat controllers / transaction script API layer`.

Why it's a problem: the HTTP boundary becomes the real home of business rules, which makes reuse hard, testing coarse, and non-HTTP entry points impossible without duplicating logic.

Better split:
- Route/controller: parse request, call one application service, map result to HTTP.
- Application service: enforce use-case rules like `CreateChatSession`, `RunRagSearch`, `UploadKnowledgeFile`, `AuthenticateUser`.
- Repositories/gateways: persistence and external systems.

2. **Business logic is mixed with infrastructure concerns**
In
[auth.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L6), auth logic reaches into Redis via `redisInstance` from the queue module. In
[rag.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L13), search flow is interleaved with Redis cache policy. In
[files.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L6), the route knows queue availability, storage provider choice, processor classes, and embedding service construction.

Likely smell: `Infrastructure leakage into domain/application flow`.

Why it's a problem: changing Redis, queue strategy, storage backend, or sync/async execution changes use-case code instead of adapters. That makes the platform brittle and discourages clean substitutions.

Better split:
- Application service depends on interfaces like `AuthSessionStore`, `SearchCache`, `DocumentIndexingPort`, `FileBlobStore`.
- Infrastructure adapters implement those interfaces for Redis, BullMQ, S3/R2/local FS, etc.
- Routes should never import queue, Redis, or storage implementations directly.

3. **Data access is mixed with orchestration**

[chat.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L20) fetches encrypted keys, decrypts them, creates sessions, counts messages, and later drives model calls.
[rag.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L137) verifies ownership, runs search, logs history, updates sessions, tracks API usage, and caches results.
[auth.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L48) reads and writes `users` and `userSecurity` while also deciding lockout and token issuance.

Likely smell: `No repository boundary / orchestration over raw tables`.

Why it's a problem: every route reconstructs its own persistence workflow. Invariants like "session updated when query logged" or "lockout increments on failed auth" are scattered and easy to break.

Better split:
- Repositories: `UserRepository`, `ChatSessionRepository`, `RagSearchRepository`, `SecurityRepository`.
- Application service coordinates them in one transaction-aware use case.
- Entities/value objects hold rules like lockout thresholds or session status transitions.

4. **Route files contain AI policy and product heuristics**

[rag.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L219) estimates embedding cost inline.
[chat.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L15) fetches/decrypts provider keys in the route.
[files.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L95) defines text chunking heuristics in the API layer.
[auth.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L59) hardcodes password policy and lockout behavior inline.

Likely smell: `Policy logic embedded in delivery layer`.

Why it's a problem: the code that should express product rules is buried in transport code, so it's harder to test, version, or reuse from workers and background jobs.

Better split:
- Move AI cost/accounting to `UsageAccountingService`.
- Move provider-key resolution to `CredentialResolutionService`.
- Move chunking and file acceptability to `KnowledgeIngestionPolicy`.
- Move auth lockout/password rules to `IdentityPolicy`.

5. **"Services" still contain infrastructure and SQL-specific behavior**

[rag-service.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/serv

ices/rag-service.ts#L1) is presented as business logic, but it is really a pgvector-specific repository plus ranking adapter.
[enhanced-rag-service.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/enhanced-rag-service.ts#L1) mixes retrieval logic with DB querying and ranking mechanics.
[models-service.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/models-service.ts#L1) is a DB query service, not a domain service.
[workflow-executor.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-executor.ts#L1) directly knows about `ragService` and `llmService`.

Likely smell: `Service layer that is really adapter layer`.

Why it's a problem: the architecture looks layered, but responsibilities are mislabeled, so the codebase still couples business use cases to storage and vendor specifics.

Better split:
- Domain/application: `RetrieveContext`, `ExecuteWorkflowNode`, `ListAvailableModels`.
- Infrastructure: `PgVectorChunkRepository`, `SqlModelCatalog`, `OpenAIProviderGateway`.
- Compose in use-case services instead of directly wiring adapters into one another.

6. **Frontend is compensating for backend response inconsistency**
[frontend authService.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/services/authService.ts#L35) normalizes `payload.user ?? payload`, and handles `accessToken || token` in multiple calls at [line 89](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/services/authService.ts#L89) and [line 111](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/services/authService.ts#L111). It also falls back to `API_ENDPOINTS.auth.changePassword || ...` at [line 165](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/services/authService.ts#L165), implying unstable API shape.

Likely smell: `Backend contract leakage into UI normalization code`.

Why it's a problem: every UI caller becomes partially responsible for backend compatibility. That increases accidental complexity and makes the UI harder to trust.

Better split:
- Stabilize backend contracts.
- Keep one frontend API adapter that maps wire DTOs to frontend DTOs.
- Contexts/components should consume normalized models only.

7. **Frontend carries backend authorization and entitlement assumptions**

[EnhancedUserRoleContext.tsx](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx#L124) contains its own module-permission map, a full `BACKEND_ROLE_PERMISSION_MAP` at [line

133](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx#L133), legacy mode translation at [line 176](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx#L176), and a fallback synthesized access snapshot at [line 237](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx#L237).

Likely smell: `Mirrored policy logic in client`.

Why it's a problem: the UI is now a second policy engine. If backend permissions change, the frontend can drift and show the wrong affordances or onboarding.

Better split:
- Backend owns access/entitlement calculation entirely.
- Frontend consumes a single access snapshot DTO.
- Client fallback should be "unknown access/loading," not reconstructed policy.

8. **Frontend carries deployment and identity shortcuts that belong elsewhere**

[AuthContext.tsx](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx#L95) contains a hardcoded support token and fabricated support user in dev mode at [line 97](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx#L97). That's environment/bootstrap behavior embedded in application state management.

Likely smell: `Operational shortcut in product runtime`.

Why it's a problem: it hides auth problems, makes testing misleading, and risks unsafe drift if that pattern survives in more environments.

Better split:
- Put dev bootstrap behind a backend seed/mock-auth endpoint or explicit local dev adapter.
- Keep AuthContext focused on session state, not environment bypasses.

9. **AI-specific logic is scattered across backend and frontend**
AI concerns are split across [llm-providers.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/llm-providers.ts#L1), [embedding-service.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/embedding-service.ts#L1), [rag-modes.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/rag-modes.ts#L1), [rag.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L15), [chat.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L3), and on the client in

[LLMContext.tsx](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/LLMContext.tsx#L37) with a large hardcoded model catalog.

Likely smell: `Scattered AI policy / no unified AI core`.

Why it's a problem: model selection rules, prompt strategy, provider capabilities, fallback behavior, and retrieval modes are spread across unrelated modules. That makes it hard to evolve the platform coherently.

Better split:
- Introduce a centralized `AI Core` or `Inference Domain` with:
  - model registry
  - provider routing
  - prompt policy
  - RAG mode policy
  - token/cost accounting
- Frontend should query model/routing metadata, not ship its own primary model catalog.

10. **The landing-page workspace duplicates product logic instead of depending on shared modules**
The landing app shell is simple, but it still contains copies of product contexts/services such as [landing auth context](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/landing-page/src/contexts/AuthContext.tsx#L1), [landing LLM context](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/landing-page/src/contexts/LLMContext.tsx#L1), and [landing ragApi](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/landing-page/src/services/ragApi.ts#L1).

Likely smell: `Responsibility duplication across apps`.

Why it's a problem: fixes to auth/API assumptions or AI behavior must be made in two apps, which guarantees drift.

Better split:
- `landing-page` should only own marketing, survey, and CTA flows.
- Shared product clients/hooks should live in a reusable package if both apps truly need them.
- Otherwise delete the duplicate product layer from `landing-page`.

11. **Domain contracts are duplicated instead of centralized**
Backend defines domain events locally in [services/backend/src/utils/domain-events.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/utils/domain-events.ts#L1), while the shared SDK defines nearly the same thing in [shared/sdk/src/events/domain-events.ts](/Users/aeishwary/JoaLLM-platform/joallm-platform/shared/sdk/src/events/domain-events.ts#L1).

Likely smell: `Split source of truth`.

Why it’s a problem: event names, payloads, and semantics can diverge silently, especially once more services appear.

Better split:
- One shared contract package owns event schemas and DTOs.
- Backend imports contracts from there.
- Local backend code should only implement dispatching, not redefine the schema.

Overall Pattern

The biggest leak is that the repo has naming that suggests layers and domains, but the runtime responsibilities still center on route files and client contexts. A cleaner split would look like:

- Controllers/routes: HTTP only
- Application services: one use case each
- Domain/policy layer: auth rules, access rules, RAG mode rules, workflow semantics
- Repositories/gateways: DB, Redis, queue, storage, provider SDKs
- Frontend API adapters: normalize wire contracts once
- Frontend state/UI: consume normalized app models only

If you want, I can turn this into a concrete refactor map next: “which files to split first, into what modules, and in what order.”